

LABORATORIO 1

Arquitectura primero: entendiendo las 5 responsabilidades

Laboratorio 1

architecture

Arquitectura Primero

Entendiendo las 5 responsabilidades de la arquitectura de software.

Actividad 1

groups

Fase AI-DLC

Inception / Mob Elaboration de manera colaborativa.

Nivel

school

Fundamentos

Hoy NO se escribe código de aplicación.

Stack de Hoy

layers

Herramientas

Diagramas (Mermaid) + Markdown + Git + GitHub.

Mapa del laboratorio (18 secciones)

Secciones 1–5

Identificación y Concepto

Por qué, concepto arquitectónico, microproyecto y resultado esperado.

Secciones 6–8

Preparación y Desarrollo

Preparación del entorno, desarrollo guiado en 10 pasos y checkpoints.

Sección 9

Solucionario Guiado

Se abre DESPUÉS del intento del equipo.

Secciones 10–13

Desafíos Críticos

Ejercicios aplicados de arquitectura, sostenibilidad, accesibilidad y seguridad.

Sección 14

Actividad AI-DLC

Trabajo colaborativo mediante mob session y registro de herramientas de IA.

Secciones 15–18

Transferencia al MVP

Consolidación de evidencias, commit obligatorio en repositorio real y reflexión.

PARTE 1 — ¿Por qué estamos aprendiendo esto?

La analogía de la casa sin plano

home La Analogía

Te piden construir una casa y lo primero que hacés es comprar ladrillos, cemento y ventanas. Sin un plano.

warning Pegás ladrillos donde te parece. A la semana:

- El baño no tiene dónde ir
- La cocina quedó sin ventana
- La escalera choca con una pared

code El Software

Con el desarrollo de software pasa exactamente lo mismo cuando se aprenden tecnologías de forma aislada sin entender la estructura general.

Herramientas Sueltas

`'React' 'Node' 'PostgreSQL'`

Construir sin arquitectura produce código desordenado e inmantenible.

El problema concreto: una app SIN capas

code Código Monolítico (Express App)

```
app.post('/tasks', (req, res) => {  
  const title = req.body.title;  
  
  if (title.length > 0) { // (a)  
    if (title.includes('grosería')) { // (b)  
      // ...  
    }  
  
    const sql = `INSERT INTO... '${title}'` // (c),  
    (d)  
  
    db.query(sql, (err, rows) => {  
      res.send(`<li>${rows[0].title}</li>`) // (e)  
    });  
  }  
});
```

warning Problemas de Acoplamiento

(a) Validación de formato: Debería ocurrir en la capa de entrada/API antes de procesar nada.

(b) Regla de negocio: Las políticas (como moderación de contenido) pertenecen exclusivamente a la Capa de Negocio.

(c) SQL y (d) Concatenación: El acceso a datos está mezclado en el controlador, exponiendo vulnerabilidades críticas de seguridad.

(e) HTML dentro de la API: La presentación visual está acoplada directamente con la lógica del backend.

La idea: separación de responsabilidades

layers La Arquitectura de Capas

¿Cómo funciona la separación de responsabilidades?

- **Organización en 5 capas:** Una aplicación web se organiza en 5 responsabilidades encadenadas.
- **Un solo trabajo:** Cada una tiene UN solo trabajo y ninguna hace el trabajo de otra.
- **Flujo secuencial:** Cada una le pasa el resultado a la siguiente capa en la cadena.
- **Separation of Concerns:** Es una idea vieja (años 70) y muy poderosa que evita el acoplamiento dañino.

Las 5 responsabilidades

01 USUARIO

→ (ve, escribe, hace clic)

02 FRONTEND

→ muestra información y captura la interacción

03 API

→ traduce peticiones HTTP en llamadas a la lógica

04 LÓGICA DE NEGOCIO

→ aplica las reglas (qué se permite, qué no, en qué orden)

05 ACCESO A DATOS (Repository)

→ traduce la intención en consultas concretas

06 BASE DE DATOS

→ guarda de forma persistente y consistente

check_circle QUÉ HACE

- **Muestra información en pantalla:** Representa visualmente la interfaz y captura clics, textos y formularios del usuario.
- **Se comunica con el servidor:** Pide datos a la API y se encarga de mostrar tanto las respuestas exitosas como los errores.

cancel QUÉ NO HACE

- **Sin lógica de negocio:** No decide las reglas operacionales ni de procesos del sistema.
- **Sin acceso directo a datos:** No sabe SQL ni habla directamente con la base de datos.
- **No persiste información:** No guarda la "verdad" de los datos, ya que esta reside exclusivamente en la base de datos.

Conexiones e Integración

HABLA CON: El **usuario** (hacia arriba, mediante la pantalla) y la **API** (hacia abajo, a través de peticiones HTTP + JSON).

check_circle QUÉ HACE

- **Recibe peticiones HTTP:** GET, POST, PATCH, DELETE.
- **Valida el FORMATO de entrada:** ¿hay title? ¿es texto? ¿no está vacío?
- **Llama a la lógica:** Devuelve una respuesta con un código de estado.

cancel QUÉ NO HACE

- **Sin reglas complejas:** No contiene las reglas complejas del negocio.
- **Sin base de datos directa:** No arma SQL.

Conexiones e Integración

HABLA CON: El **Frontend** (hacia arriba) y la **Lógica de negocio** (hacia abajo).

check_circle QUÉ HACE

- **Aplica las reglas:** qué se permite, qué no, en qué orden, con qué condiciones.
- **Coordina los pasos:** de una operación (normaliza datos, decide, llama al repositorio).

cancel QUÉ NO HACE

- **Desconoce el cliente:** No sabe si la llamó HTTP o una app de escritorio.
- **Desconoce el origen:** No sabe si los datos vienen de PostgreSQL, de un archivo o de memoria.

Conexiones e Integración

HABLA CON: La **API** (hacia arriba) y el **Acceso a datos** (hacia abajo, por una interfaz).

Ficha 4 — ACCESO A DATOS (Repository)

check_circle QUÉ HACE

- **Traduce 'guardá esta tarea' / 'traeme las tareas':** en operaciones concretas contra la base.
- **Aísla los detalles:** del motor de datos (si es PostgreSQL, MySQL, etc.).

cancel QUÉ NO HACE

- **No toma decisiones:** del negocio.
- **¿Se puede reabrir?:** '¿Se puede reabrir esta tarea?' NO es su problema.

Conexiones e Integración

HABLA CON: La **Lógica de negocio** (hacia arriba) y la **Base de datos** (hacia abajo, por SQL).

check_circle QUÉ HACE

- Guarda los datos: de forma **PERSISTENTE** (sobreviven a un reinicio).
- Garantiza consistencia: tipos, claves primarias, unicidad, relaciones.

cancel QUÉ NO HACE

- No ejecuta la lógica: de la aplicación. No decide flujos. No renderiza nada.

Conexiones y Tecnología

HABLA CON: solo el **Acceso a datos**.

En este curso: PostgreSQL, provisto por **Supabase** (base gestionada en la nube).

Las 3 preguntas que definen una arquitectura

Para CADA capa, siempre respondé:

01

¿Qué problema resuelve?

02

¿Qué responsabilidad tiene
— y cuál NO?

03

¿Cómo le pasa el trabajo a
la siguiente capa?

Conclusión Clave

Si podés responder las 3 para las 5 capas, entendés la arquitectura.

Por qué separar reduce la complejidad

psychology

Se entiende sola

Menos cosas en la cabeza a la vez al concentrarse en un único dominio.

fact_check

Se prueba sola

Permite realizar pruebas unitarias y de integración de manera aislada y focalizada.

swap_horiz

Se cambia sola

El impacto de las modificaciones queda confinado a su propia capa:

- **PostgreSQL por MySQL:**
solo cambia Acceso a datos.
- **React por Vue:**
solo cambia Frontend.

bug_report

Errores acotados

La falla no se propaga de forma catastrófica. Un bug en la presentación no corrompe la base de datos.

Arquitectura primero, tecnología después

Responsabilidad	Implementación en este curso	Alternativas válidas
Frontend	React + Vite	Vue, Svelte, HTML+JS
API	Node.js + Express	Fastify, Slim PHP, FastAPI
Lógica de negocio	módulos JS (service)	cualquier lenguaje; el patrón no cambia
Acceso a datos	Repository con pg	Prisma, Sequelize, otro driver
Base de datos	PostgreSQL (Supabase)	PostgreSQL propio, MySQL, SQLite
Empaquetado	Docker + Compose	ejecución nativa, Podman

PARTE 2 — El microproyecto: Mini Task Manager

Mini Task Manager (conceptual)

Concepto Base

Aplicación mínima para gestionar tareas. **NO es el producto final** del equipo: es el andamio.

Funcionalidad del MVP

- 1 **Ver la lista de tareas**
- 2 **Crear una tarea (con un título)**
- 3 **Cambiar el estado (pending ↔ done)**
- 4 **Eliminar una tarea**

Entidad Task (Atributos)

id Identificador único (PK)
title Título o descripción
status Estado de la tarea
created_at Fecha de creación
updated_at Última modificación

Flujo 'Crear una tarea' por las 5 capas

Paso 1

Interacción

1. Usuario

Escribe 'Comprar pan' y hace clic en Agregar.

2. Frontend

Envía `POST /tasks` con el título.

Pasos 3-5

Validación

3. API (Filtro)

Valida el formato (¿existe y no está vacío?). Responde 400 si falla.

4–5. Lógica

La API delega a la Lógica de Negocio, que normaliza la tarea y define el estado inicial.

Pasos 6-8

Persistencia

6–8. Base de Datos

Lógica → Repository → `INSERT`

La base de datos procesa la consulta SQL y retorna los datos generados automáticamente (ID y fechas).

Pasos 9-11

Respuesta

9–11. Retorno

El objeto mapeado sube: Repository → Lógica → API responde con éxito (`201 Created`).

Pasos 12-13

Cierre

12–13. Renderizado

El Frontend recibe la respuesta, inserta la nueva tarea directamente en la interfaz visual y el usuario la ve inmediatamente.

Resultado esperado: ANTES → DESPUÉS

✗ ANTES

Enfoque Inicial

Idea vaga de 'una app tiene frontend y backend'. Sin repositorio organizado. Sin diagramas de arquitectura.

Falta de Definición

Ante la pregunta "*¿Dónde va esta regla de negocio?*" la respuesta común es un impreciso "*en el backend*" (demasiado vago).

✓ DESPUÉS

Estructura y Responsabilidades

- Diagrama claro de las **5 responsabilidades** del Mini Task Manager (Mermaid, documentado en el repo).
- Tabla de responsabilidades completa junto al diagrama del **MVP real** diseñado por el equipo.

Sostenibilidad y Control

- Presupuesto de sostenibilidad + repositorio formal en GitHub + Registro de uso de IA.

Preparación del entorno

Herramienta	¿Qué es / Para qué sirve?	Verificación / Comando	Resultado esperado
Git	Control de versiones local y gestión del código fuente	<code>git --version</code>	'git version 2.x'
GitHub	Plataforma de hosting y repositorio remoto para colaboración	Iniciar sesión en el navegador	Perfil visible públicamente
VS Code	Editor de código extensible para desarrollo	Abrir carpeta de trabajo	Explorador de archivos visible
Node.js + npm	Entorno de ejecución y gestor de paquetes Uso desde Lab 3	<code>node --version</code> <code>npm --version</code>	v20.x / 10.x
Mermaid	Generación de diagramas y gráficos a partir de texto plano	Ingresar a <code>mermaid.live</code>	El diagrama se dibuja correctamente

PARTE 3 — Desarrollo guiado (Día 1: pasos 1–4)

Pasos 1–4 (Día 1)

PASO 1

Experimento mental:
analizar la 'app sin
capas' y listar cuántas
cosas hace

PASO 2

Completar la tabla
'qué hace / qué NO
hace / con quién
habla' para las 5 capas

PASO 3

Escribir el diagrama
Mermaid del Mini
Task Manager y verlo
en `mermaid.live`

PASO 4

Escribir el flujo 'Crear
una tarea' en ~13
pasos por las 5 capas

flag CHECKPOINT Día 1 al final

PARTE 4 — Desarrollo guiado (Día 2: pasos 5–10)

Pasos 5–7 (Día 2): del problema al MVP y al presupuesto

PASO 5

[docs/alcance.md](#)

Definición del Alcance

- Identificar el **problema** central y el **usuario** objetivo.
- Definir un **MÁXIMO de 4 acciones** clave para el MVP.
- Crear la lista de exclusiones (Filosofía **[YAGNI]**: no lo vas a necesitar).

PASO 6

[docs/arquitectura/03-mvp.md](#)

Diseño de Arquitectura

- Diseñar el **diagrama de 5 capas** adaptado al dominio del equipo.
- Definir **+1 regla de negocio propia** de alta prioridad.

"Cuando pasa X, no se permite Y sin Z"

PASO 7

[docs/sostenibilidad/presupuesto.md](#)

Presupuesto y Límites

- Establecer **límites objetivo de peso** y volumen de requests.
- Control estricto de **dependencias** externas.

Regla del equipo: antes de agregar una dependencia, responder 3 preguntas por escrito.

PASO 8 (Día 2): crear el repositorio y la estructura

FASE 1

Inicialización

Crear el Entorno

- `mkdir mvp-mi-equipo`
Crea la carpeta principal del proyecto.
- `cd mvp-mi-equipo`
Accede al directorio recién creado.
- `git init`
Inicializa el repositorio Git (crea carpeta oculta `.git`).

FASE 2

Estructura

Árbol de Carpetas

- `mkdir -p docs/arquitectura docs/sostenibilidad docs/ia api frontend backend reports`
Crea la estructura del proyecto en un solo comando:

```
├── docs/ (arq, sostenib, ia)
├── api/ & backend/
├── frontend/
└── reports/
```

FASE 3

Configuración

Archivos Clave

- `.gitignore`
Evita subir archivos pesados o sensibles al repositorio:
`node_modules/`, `.env`,
`dist/`.
- `README.md`
Documento de bienvenida con el equipo, arquitectura, stack tecnológico y estado.

PASO 9 (Día 2): primer commit y push

FASE 1: LOCAL

Preparar & Capturar

Registrar Cambios

```
git add .
```

Prepara TODOS los cambios para la próxima 'foto'.

```
git commit -m '...'
```

Saca la foto (mensaje: qué y por qué, no cómo).

FASE 2: CONEXIÓN

Destino & Rama

Vincular Servidor

```
git remote add origin  
https://github.com/...
```

Asocia tu repositorio local con el servidor de GitHub.

```
git branch -M main
```

Renombra la rama principal a 'main'.

FASE 3: PUBLICAR

Subida & Control

Sincronizar y Ver

```
git push -u origin  
main
```

Sube la rama al remoto; -u guarda la relación.

```
git log --oneline
```

Verificar: 1 línea con el hash y el mensaje de confirmación.

PASO 10 (Día 2): Mob session + Registro de IA

PASO 01

El Facilitador

Define la pregunta inicial (ej.:
'¿qué estructura de carpetas para el backend?')

PASO 02

El Piloto

Se la hace a la IA para formular el prompt inicial y explorar opciones.

PASO 03

La IA

Propone algo como base de partida técnica o solución preliminar.

PASO 04

El Retador

Cuestiona críticamente: *'¿esto hace visibles las 5 capas? ¿es simple? ¿lo necesitamos?'*

PASO 05

El Equipo

DECIDE colectivamente con argumento técnico propio, validando la arquitectura.

PASO 06

El Escriba

Completa la bitácora registrando la decisión en:
[docs/ia/registro-ia.md](#)

Checkpoints (consolidado)



Responsabilidades

El equipo nombra las 5 responsabilidades sin apuntes



Matriz de Comunicación

La tabla 'qué hace / qué NO hace / con quién habla' está completa



Diagrama Mermaid

Se dibuja en mermaid.live y se renderiza en GitHub



Flujo de Tareas

El flujo 'crear una tarea' está escrito en ~13 pasos



Repositorio Remoto

El repo está en GitHub con la estructura y el .gitignore correcto (incluye .env)



Registro de Decisiones

docs/ia/registro-ia.md tiene ≥ 2 decisiones + 'Qué rechazamos'



Historial de Git

git log --oneline muestra el commit del Lab 1

Desafío de pensamiento arquitectónico

DESAFÍO 01

¿Qué pasa si ponemos SQL en el componente React?

(seguridad, cambio de base, pruebas, credenciales)

DESAFÍO 02

¿Qué pasa si el Controller contiene TODAS las reglas?

(reutilización, tamaño, pruebas)

DESAFÍO 03

¿Qué pasa si mañana cambiamos PostgreSQL por MySQL?

¿Qué capa se toca? ¿Cuál NO?

DESAFÍO 04

¿Qué parte de la arquitectura PERMANECE aunque cambiemos...?

React + Express + PostgreSQL a la vez

DESAFÍO 05

Tu MVP: 1 funcionalidad

¿En qué capa vive cada parte?

Desafío de sostenibilidad

Decisión de hoy: diagramas en Mermaid / Markdown en lugar de Figma o Miro



Peso y Eficiencia

~2 KB

de texto plano

~~Varios MB + cuenta~~



Control de Versiones

Versionable con Git

SÍ (Control total)

~~NO es posible en la nube~~



Disponibilidad

Soporte Offline

SÍ (Trabaja sin conexión)

~~NO (Requiere internet)~~



Trazabilidad

Diff Legible

SÍ (Línea por línea)

~~IMPOSIBLE comparar~~



Pregunta Obligatoria

¿Qué recurso estamos ahorrando con esta arquitectura?

Decisión de Arquitectura

Aunque hoy no hay interfaz, la decisión queda sentada:



Frontend

Responsable de la accesibilidad final en el navegador (HTML semántico, manejo de foco, contraste de color y navegación por teclado).



API (Backend)

Responsable de procesar datos y devolver mensajes de error CLAROS, descriptivos y en formato de texto legible para lectores de pantalla.

Comparación de Errores

Caso de validación al enviar POST /tasks con título vacío:



MAL (Insuficiente)

```
{ error: 'ERR_4021' }
```

No aporta contexto. El usuario final o lector de pantalla no sabrá cómo corregirlo.



BIEN (Accesible)

```
{ error: 'El título no puede estar vacío.', field: 'title' }
```

Mensaje claro y directo. Permite vincular el error visualmente al input correspondiente.

Desafío de seguridad: inyección SQL

Vulnerabilidad en App sin Capas

Concatenar valores directamente en la consulta SQL:

<> Código Vulnerable

```
const sql = `INSERT ... VALUES ('${title}', 'pending')`
```

🚫 Inyección de Código

```
'); DROP TABLE tasks;--
```

La cadena se ejecuta como SQL directo en la base de datos (**OWASP A03:2021**).

Soluciones de Arquitectura

Mitigación de riesgos y manejo de secretos:

🛡️ Consultas Parametrizadas

Se resuelve en la capa de **Acceso a Datos** utilizando marcadores de posición (**\$1**, **\$2**).

Implementado en el Lab 7.

🔑 Gestión de Credenciales

Regla de seguridad: Los secretos nunca se suben al repositorio público.

Contraseña de la base + claves de Supabase deben configurarse localmente en un archivo **.env**

Transferencia al MVP real

Qué reutilizás TAL CUAL

Elementos reutilizables inmediatamente:

- **Patrón de 5 responsabilidades** y las fronteras establecidas.
- La regla de oro: "**cada capa habla solo con su vecina**".
- **Presupuesto de sostenibilidad** para el control del ciclo de vida.
- **Estructura de carpetas** organizada por capas del proyecto.
- **Registro de IA** para trazar las decisiones y generación de código.

Qué arquitectura CONSERVÁS

Estructura inalterable de 5 capas:

1. Frontend
2. API
3. Lógica
4. Acceso a datos
5. Base de datos

Qué CAMBIA en el MVP

Adaptación al dominio real:

- **La Entidad de Negocio:**
Task → **tu-entidad**
- **Las Acciones y Reglas:**
Flujos lógicos, validaciones y reglas particulares de tu propio negocio.
- **Los Endpoints de la API:**
/tasks → **/tu-recurso**

Evidencia para la Actividad 1 + commit obligatorio

folder Evidencias en el Repo

Actividad 1

'Del problema del entorno al alcance del MVP y la arquitectura'

Archivos requeridos

- **Problema y MVP:**
`docs/alcance.md`
- **Arquitectura:**
`docs/arquitectura/01,02,03`
- **Sostenibilidad:**
Presupuesto de sostenibilidad
- **Mob Elaboration:**
`registro-ia.md`
- **Repositorio:** URL oficial

photo_camera Captura Obligatoria

Ruta del archivo

`reports/lab-01-repo.png`

Contenido requerido

La captura debe mostrar claramente:

- El repositorio en **GitHub** con toda la estructura de carpetas creada.
- La confirmación del **commit obligatorio** correspondiente a la entrega.

terminal Commit Obligatorio

Secuencia de Comandos

Ejecuta estos pasos en tu terminal:

```
$ git status
$ git add .
$ git commit -m "...
$ git push
```

Asegúrate de documentar correctamente el mensaje del commit para mantener la trazabilidad del proyecto.

Reflexión metacognitiva + Decisión tecnológica

Reflexión Metacognitiva

Entregable Individual

docs/reflexiones/lab-01-<nombre>.md (5 líneas c/u)

Ejes de Análisis Obligatorios:

- **Dimensión Técnica y de Equipo:**
Responsabilidad aprendida · Tecnología usada · ¿Otra tecnología? · Decisión final del equipo.
- **Dimensión de IA y Sostenibilidad:**
Qué propuso la IA · Qué rechazamos · Recurso reducido · Mejora de accesibilidad · Reutilización en el MVP.

Decisión Tecnológica Justificada

Análisis de Sostenibilidad

Tabla de 7 preguntas clave

Pregunta Conductora:

"¿Qué es aquello que permanece inalterable aunque decidas cambiar de tecnología en el futuro?"

Foco de la Justificación:

- Las 5 responsabilidades
- Definición precisa de sus fronteras

Rúbrica de evaluación (modelo socioformativo · UPDS)

Estructura y Niveles

Evaluación sobre 100 puntos:

4 criterios × 25 puntos cada uno

Niveles de Desempeño:

Preformal (10–13)

Receptivo (14–16)

Resolutivo (17–19) — *Mínimo para aprobar*

Autónomo (20–23)

Estratégico (24–25)

4 Criterios de Evaluación

C1 — Comprensión de la arquitectura por capas

5 responsabilidades y sus fronteras.

C2 — Aplicación al MVP

Alcance recortado + presupuesto de sostenibilidad.

C3 — Integración ética y crítica de la IA

Registro de IA: la IA propone, el equipo decide.

C4 — Evidencias, trazabilidad y reflexión

Repositorio (repo), commit y documentación.

gavel Para aprobar: al menos nivel Resolutivo en los 4 criterios.

Cierre y próximo laboratorio

Lo que construiste hoy

El PLANO del MVP:

- Diseño de las **5 responsabilidades** clave.
- Definición clara de sus **fronteras**.
- Tu primera arquitectura de software estructurada.

"La arquitectura permanece. La tecnología puede cambiar."

Entregables Obligatorios

Evidencias en GitHub:

Todo el trabajo debe subirse estructurado bajo el directorio principal:

docs/

Documentos clave incluidos:

- Arquitectura y Sostenibilidad
- Registro de IA y Reflexiones
- **Mínimo 1 commit en el repositorio**

Próximo Laboratorio

LAB 2 (Día 3):

"Del problema al MVP y al CONTRATO DE LA API"

- **Backlog priorizado** para delimitar con exactitud el alcance técnico.
- Diseño formal de los endpoints clave de la API:
GET/POST/PATCH/DELETE /tasks
¡Diseñados ANTES de programar!